LAB ASSESSMENT 3 (LA3) SOFT-RECORD
BCSE203E: WEB PROGRAMMING LAB
PROGRAMME: B.TECH ; BRANCH: CSE
CLASS NUMBER: VL2025260504864;
SLOT: L33+L34/L59+L60
TIME: 15:51 – 17:30/17:40 – 19:20
VENUE: PRP232

REG NO:24BCE0686
NAME: MAYUR R KENI

School of Computer Science and Engineering

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

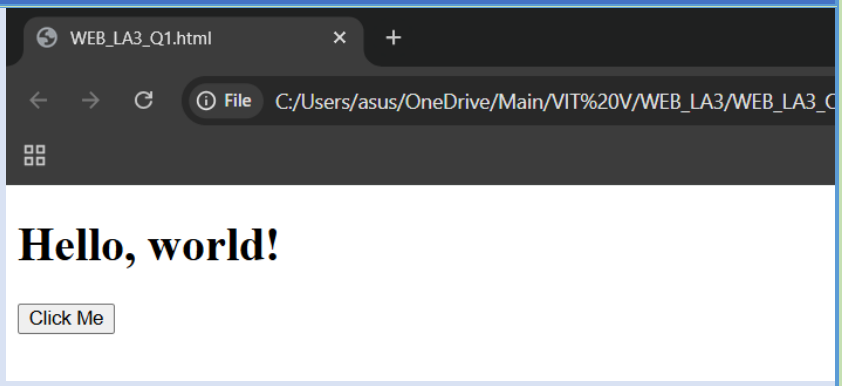
Assessment: Date:	Problem Statement-1
	Create a simple web page that displays "Hello, world!" when a button is clicked. Use JavaScript function, DOM manipulation with getElementById, and outerHTML property. Concepts: Variables (var) Function declaration DOM access (document.getElementById) outerHTML property Event handling (onclick)

List of tags used

Tag/Property	Description
	Acts as a container to hold the message and button.
<button>	Creates the clickable element to trigger the function.
<script>	Contains the JavaScript logic for the page.
onclick	Event handler that triggers the displayHello function.
document.getElementById()	Method to access the specific HTML element by its ID.
.outerHTML	Property used to replace the element and its content with a new tag.

Program

```
<!DOCTYPE html>
<html>
<body>
  <div id="container">
    <p id="msg">Initial Text</p>
    <button onclick="displayHello()">Click Me</button>
  </div>
  <script>
  function displayHello() {
    var element = document.getElementById("msg");
    element.outerHTML = "<h1 id='msg'>Hello, world!</h1>";
  }
  </script>
```

	</body> </html>
Output Screenshots	
Observations	When the button is clicked, the JavaScript swaps the <p> tag for an <h1> using outerHTML, changing both the content and the styling of the text from a plain paragraph to a large heading. The id="msg" is kept on the new element so the page can still reference it if needed.
References	

Assessment: Date:	Problem Statement-2
	Create a web page with three different buttons that change the background color to red, green, and blue respectively when clicked. Concepts: Button controls (type="button") Function with parameters document.body.style.backgroundColor Event handlers (onclick)

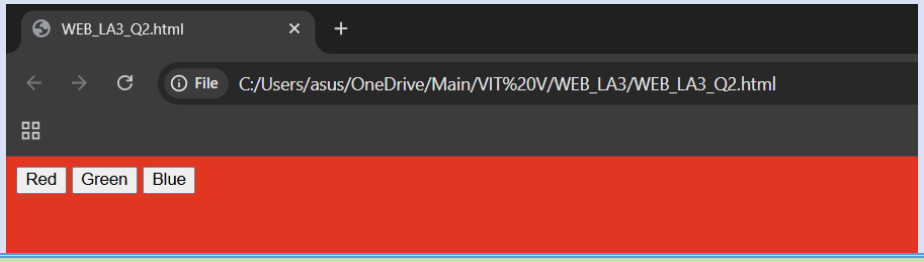
List of tags used

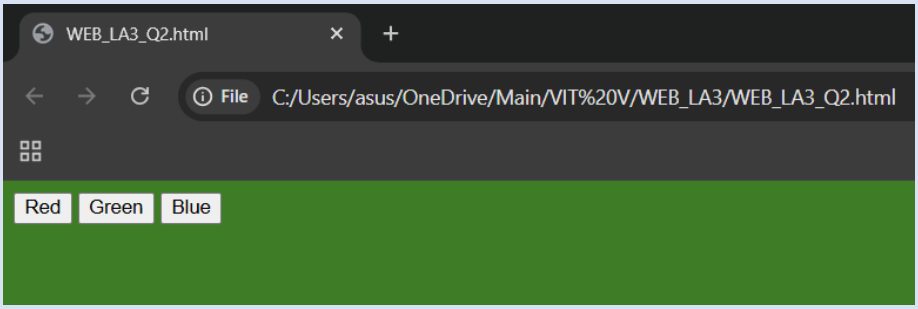
Tag/Property	Description
<button type="button">	Specifically defines a standard push button.
style.backgroundColor	Accesses the CSS background-color property via JS.
onclick="changeColor('...')"	Passes a string parameter (color) to the function.

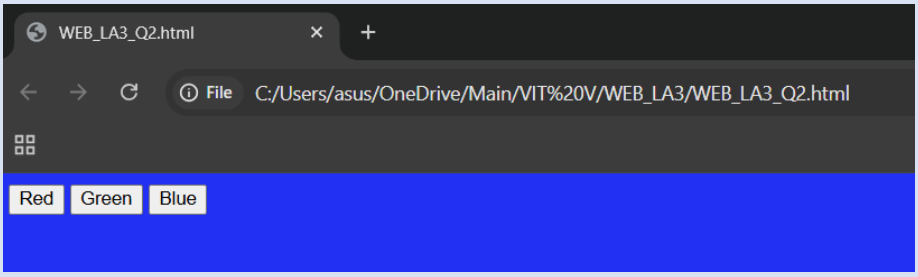
Program

```
<!DOCTYPE html>
<html>
<body>
   <button type="button" onclick="changeColor('red')">Red</button>
   <button type="button" onclick="changeColor('green')">Green</button>
   <button type="button" onclick="changeColor('blue')">Blue</button>
   <script>
   function changeColor(color) {
      document.body.style.backgroundColor = color;
   }
   </script>
</body>
</html>
```

Output Screenshots







Observations	This code illustrates how to pass arguments to JavaScript functions. Each button sends a different color string to the changeColor function, which then directly modifies the backgroundColor property of the document.body object to update the UI.
References	

Assessment: Date:	Problem Statement-3
	Create an Email Address Generator web page that takes first name and last name as input, generates an email address in the format "firstname.lastname@park.edu", displays it, clears the form, and sets focus back to the first name field. Concepts: form.elements access String concatenation (+ operator) innerHTML reset() method focus() method this.form

List of tags used

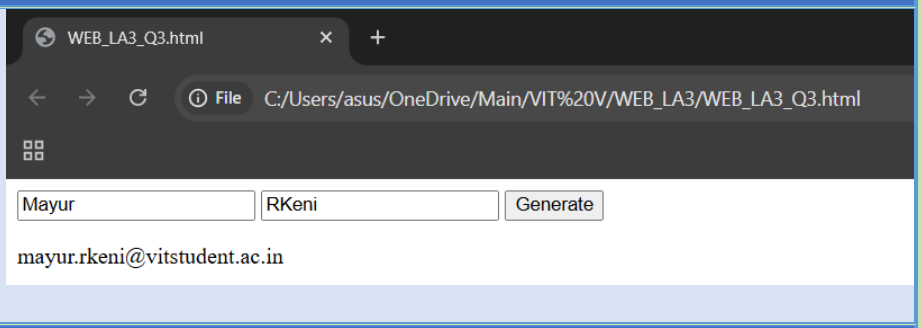
Tag/Property	Description
<input type="text">	Provides text fields for user data entry.
<p id="emailResult">	Placeholder tag to display the generated output.
.value	Retrieves the content currently inside the input field.
.toLowerCase()	Formats the input strings to lowercase for the email address.

Program

```
<!DOCTYPE html>
<html>
<body>
   <input type="text" id="fname" placeholder="First Name">
   <input type="text" id="lname" placeholder="Last Name">
   <button onclick="generateEmail()">Generate</button>
   <p id="emailResult"></p>
   <script>
   function generateEmail() {
      var fname = document.getElementById("fname").value.toLowerCase();
      var lname = document.getElementById("lname").value.toLowerCase();
      document.getElementById("emailResult").innerHTML = fname + "." + lname + "@vitstudent.ac.in";
   }
```

```
    </script>
</body>
</html>
```

Output Screenshots



Observations

The functionality focuses on string manipulation and data retrieval. It extracts user input values, converts them to lowercase for standardization, and uses string concatenation (+) to build a complete email address before injecting it into the page using innerHTML.

References

Assessment: Date:	Problem Statement-4
	Write a JavaScript program that calculates the area of a rectangle. Use proper variable naming conventions (camelCase) and demonstrate loose typing by first calculating with integers, then with floating-point numbers. Concepts: Variable declaration (var) camelCase naming convention Loose typing (dynamic typing) Integer and float calculations innerHTML

List of tags used

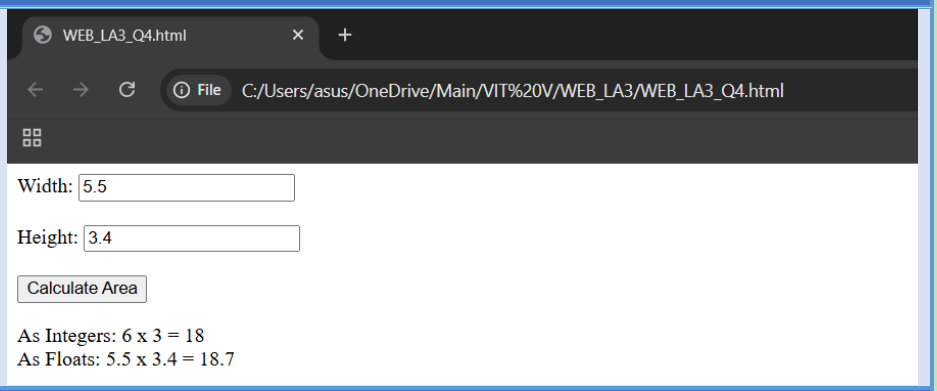
Tag/Property	Description
step="0.1"	Attribute allowing the numeric input to accept decimal values.
Math.round()	Function used to convert input to the nearest integer.
parseFloat()	Function that parses a string and returns a floating-point number.
var	Keyword used to declare variables using camelCase naming.
.innerHTML	Property to display both the integer and float results on the page.

Program

```
<!DOCTYPE html>
<html>
<body>
   <div id="rectContainer">
      Width: <input type="number" id="userWidth" step="0.1"><br><br>
      Height: <input type="number" id="userHeight" step="0.1"><br><br>
      <button onclick="calculateArea()">Calculate Area</button>
      <p id="resultDisplay"></p>
   </div>
   <script>
   function calculateArea() {
      var rectWidth = document.getElementById("userWidth").value;
      var rectHeight = document.getElementById("userHeight").value;
```

```
        var intWidth = Math.round(rectWidth);
        var intHeight = Math.round(rectHeight);
        var integerArea = intWidth * intHeight;

        var floatWidth = parseFloat(rectWidth);
        var floatHeight = parseFloat(rectHeight);
        var floatArea = floatWidth * floatHeight;

        document.getElementById("resultDisplay").innerHTML =
            "As Integers: " + intWidth + " x " + intHeight + " = " +
integerArea + "<br>" +
            "As Floats: " + floatWidth + " x " + floatHeight + " = " +
floatArea;
    }
    </script>
</body>
</html>
```

Output Screenshots



Observations

This program illustrates how JavaScript manages different numerical representations. By using the step attribute, the user can provide precise dimensions. The script then demonstrates **loose typing** by processing those inputs as both rounded integers (via Math.round) and decimal floats (via parseFloat), displaying both outcomes simultaneously to show the difference in precision.

References

Assessment: Date:	Problem Statement-5
	Create a calculator that takes two numbers as input, performs addition, subtraction, multiplication, or division based on button clicked, and displays the result. Use number input type. Concepts: Number control (type="number") step="any" attribute parseFloat() function switch statement Division by zero handling

List of tags used

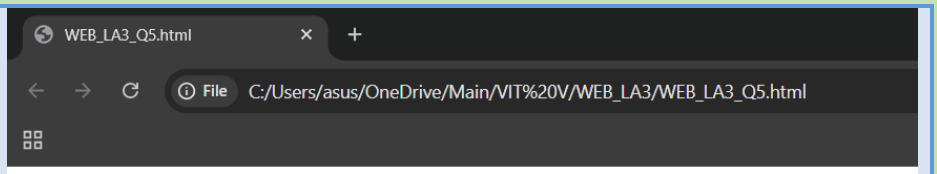
Tag/Property	Description
type="number"	Specifies that the input field should only accept numeric values.
step="any"	Allows the number input to accept any integer or floating-point value without validation errors.
parseFloat()	A JavaScript function that converts the string input from the fields into actual decimal numbers.
switch	A control statement used to execute different blocks of code based on which operator button was clicked.
if (num2 === 0)	Logical check used to handle division by zero and prevent "Infinity" results.

Program

```
<!DOCTYPE html>
<html>
<body>
  <div class="calculator">
    <h2>Simple Calculator</h2>
    <input type="number" id="num1" step="any" placeholder="Number 1">
    <input type="number" id="num2" step="any" placeholder="Number 2"><br><br>

    <button onclick="calculate('+')">+</button>
    <button onclick="calculate('-')">-</button>
    <button onclick="calculate('*')">*</button>
    <button onclick="calculate('/')">/</button>
```

```
        <p id="calcResult">Result: </p>
    </div>

    <script>
    function calculate(operator) {
        var n1 = document.getElementById("num1").value;
        var n2 = document.getElementById("num2").value;
        var result;

        var num1 = parseFloat(n1);
        var num2 = parseFloat(n2);

        switch (operator) {
            case '+':
                result = num1 + num2;
                break;
            case '-':
                result = num1 - num2;
                break;
            case '*':
                result = num1 * num2;
                break;
            case '/':
                if (num2 === 0) {
                    result = "Error: Division by zero";
                } else {
                    result = num1 / num2;
                }
                break;
            default:
                result = "Invalid Operation";
        }

        document.getElementById("calcResult").innerHTML = "Result: "
+ result;
    }
    </script>
</body>
</html>
```

Output Screenshots

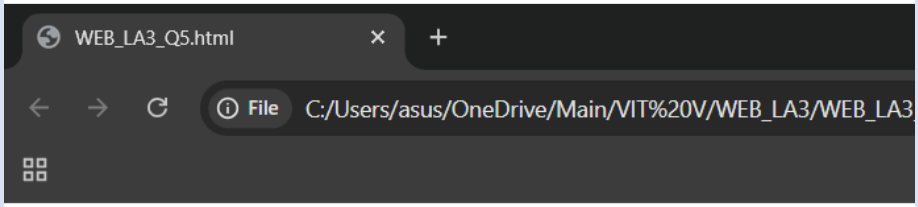


Simple Calculator

44.4 22.2

+ - * /

Result: 66.6

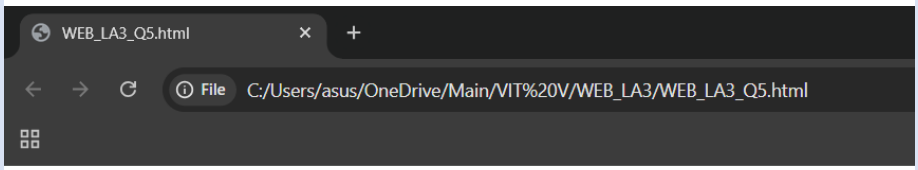


Simple Calculator

44.4 22.2

+ - * /

Result: 22.2

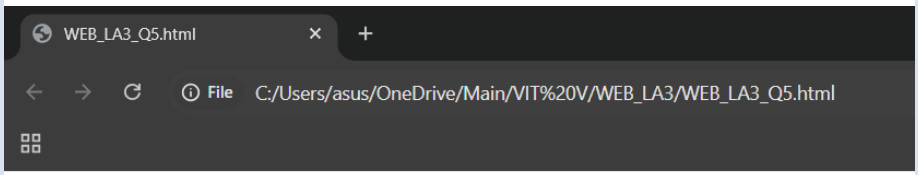


Simple Calculator

44.4 22.2

+ - * /

Result: 985.68



Simple Calculator

44.4 22.2

+ - * /

Result: 2

Observations	This program demonstrates the use of a switch statement to manage multiple operations within a single function. By using step="any", the calculator remains flexible for both whole numbers and decimals. The logic also includes a safety check for division by zero, ensuring the application provides a user-friendly error message instead of a mathematical error.
References	

Assessment: Date:	Problem Statement-6
	Create a comprehensive student information system that demonstrates: buttons, functions, variables, identifiers, assignment statements, objects, DOM manipulation, form handling, various controls, and the reset/focus methods. Concepts: JavaScript objects Object methods (this keyword) Arrays Dynamic table generation Global variables Function parameters CSS styling

List of tags used

Tag/Property	Description
<form>	Defines an HTML form that collects user input to be processed by JavaScript.
<fieldset>	Draws a box around the form elements to group them visually.
<legend>	Provides a caption/title for the <fieldset> group.
<label>	Defines a text label for an input element, improving accessibility.
<input>	The field where the user types data (Name, Reg No, Dept).
<button>	A clickable element used to trigger the save or reset functions.
<table>	The container for displaying the registered student data in rows and columns.
<thead>	Groups the header content (<th>) in a table.
<tbody>	Groups the body content in a table; used here to target where new rows are inserted.
<tr>	Defines a table row.
<th>	Defines a header cell in a table (typically bold and centered).
<td>	Defines a standard data cell in a table row.

Object	A collection of related data and functions stored in a single variable.
this	A keyword used inside the object to access its own properties (e.g., this.sName).
insertRow()	A DOM method used to add a new <tr> into the table's <tbody>.
insertCell()	A DOM method used to add a new <td> into the current row.
reset()	A form method that clears all input fields.
focus()	Moves the cursor back to the first input field automatically.

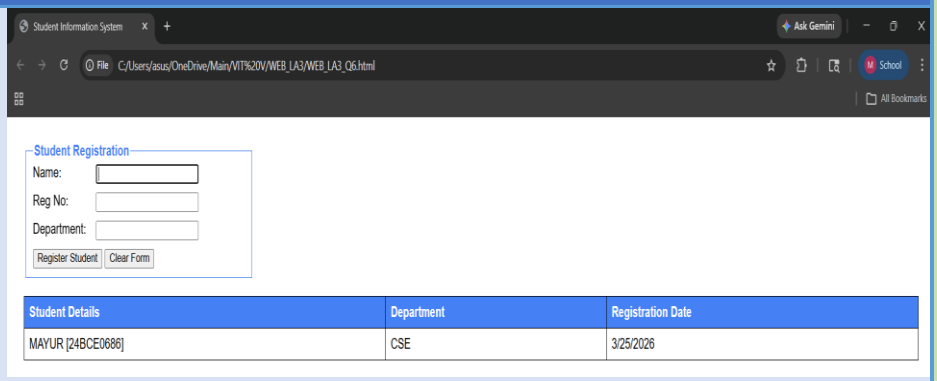
Program

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Student Information System</title>
    <style>
    body { font-family: sans-serif; padding: 20px; }

    fieldset { width: 350px; border: 1px solid #007bff; }

    legend { font-weight: bold; color: #007bff; }

    .form-group { margin-bottom: 10px; }

    label { display: inline-block; width: 100px; }

    table { width: 100%; border-collapse: collapse; margin-top: 20px; }

    th, td { border: 1px solid black; padding: 8px; text-align: left; }

    th { background-color: #007bff; color: white; }
</style>
    <script>
        var studentRegistry = [];

        function saveStudentRecord() {
            var name = document.getElementById("studentName").value;
            var regNo = document.getElementById("regNo").value;
            var dept = document.getElementById("department").value;
```

```
            if (name === "" || regNo === "") {
                alert("Please fill in required fields.");
                return;
            }

            var student = {
                sName: name,
                sReg: regNo,
                sDept: dept,
                getDisplayInfo: function() {
                    return this.sName + " [" + this.sReg + "]";
                }
            };

            studentRegistry.push(student);
            renderTable(student);
            resetForm();
        }

        function renderTable(obj) {
            var table = document.getElementById("studentTable").getElementsByTagName('tbody')[0];
            var row = table.insertRow();

            row.insertCell(0).innerHTML = obj.getDisplayInfo();
            row.insertCell(1).innerHTML = obj.sDept;
            row.insertCell(2).innerHTML = new Date().toLocaleDateString();
        }

        function resetForm() {
            document.getElementById("stuForm").reset();
            document.getElementById("studentName").focus();
        }
    </script>
</head>
<body>
    <fieldset>
        <legend>Student Registration</legend>
        <form id="stuForm">
            <div class="form-group">
                <label>Name:</label>
                <input type="text" id="studentName">
            </div>
            <div class="form-group">
```

```
            <label>Reg No:</label>
            <input type="text" id="regNo">
        </div>
        <div class="form-group">
            <label>Department:</label>
            <input type="text" id="department">
        </div>
        <div class="actions">
            <button type="button" onclick="saveStudentRecord()">Register Student</button>
            <button type="button" onclick="resetForm()">Clear Form</button>
        </div>
    </form>
  </fieldset>

  <table id="studentTable">
    <thead>
        <tr>
            <th>Student Details</th>
            <th>Department</th>
            <th>Registration Date</th>
        </tr>
    </thead>
    <tbody></tbody>
  </table>
</body>
</html>
```

Output Screenshots



Observations

This application demonstrates how different HTML structural tags like **<thead>**, **<tbody>**, and **<tr>** work together with JavaScript to organize data. By utilizing **DOM methods** such as **insertRow()** and **insertCell()**, the program avoids re-rendering the entire table, making the update efficient. The inclusion of **reset()** and **focus()** provides a professional

	form-handling experience by managing the state of the UI elements after each submission.
References	

Assessment: Date:	Problem Statement-7
	Write a JavaScript program to print numbers from 1 to N using while loop. Take N as input from user. Concepts: While loop execution with condition checking Counter initialization and increment Taking user input from text field Displaying output in HTML element

List of tags used

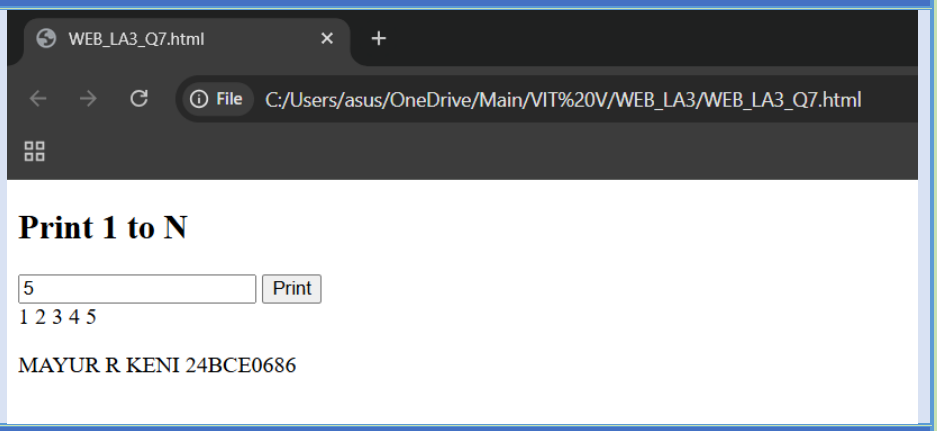
Tag/Property	Description
<input type="number">	Ensures the user provides a numeric limit for the sequence.
while	A loop structure that executes as long as the counter is less than or equal to N.
Counter	A variable (usually i) used to track the current step in the loop.
++	The increment operator that adds 1 to the counter in every cycle.
.innerHTML	The property used to output the generated sequence to the page.

Program

```
<!DOCTYPE html>
<html>
<body>
  <h2>Print 1 to N</h2>
  <input type="number" id="nVal">
  <button onclick="runLoop()">Print</button>
  <div id="out"></div>
  <script>
  function runLoop() {
    var n = parseInt(document.getElementById("nVal").value);
    var i = 1, res = "";
    while(i <= n) {
      res += i + " ";
      i++;
    }
    document.getElementById("out").innerHTML = res;
  }
  </script>
  <p>MAYUR R KENI 24BCE0686</p>
```

```
</body>
</html>
```

Output Screenshots



Observations

This program demonstrates the basic mechanics of a **while loop**. By initializing a counter at 1 and incrementing it until it reaches the user-defined limit, the script builds a string of numbers. This illustrates how JavaScript can automate repetitive tasks based on dynamic user input.

References

Assessment: Date:	Problem Statement-8
	Write a JavaScript program to print multiplication table of a number using do-while loop. Take number as input from user. Concepts: Do-while loop execution (body first, condition later) Minimum one execution guarantee Building string output with HTML line breaks Multiplication operation inside loop

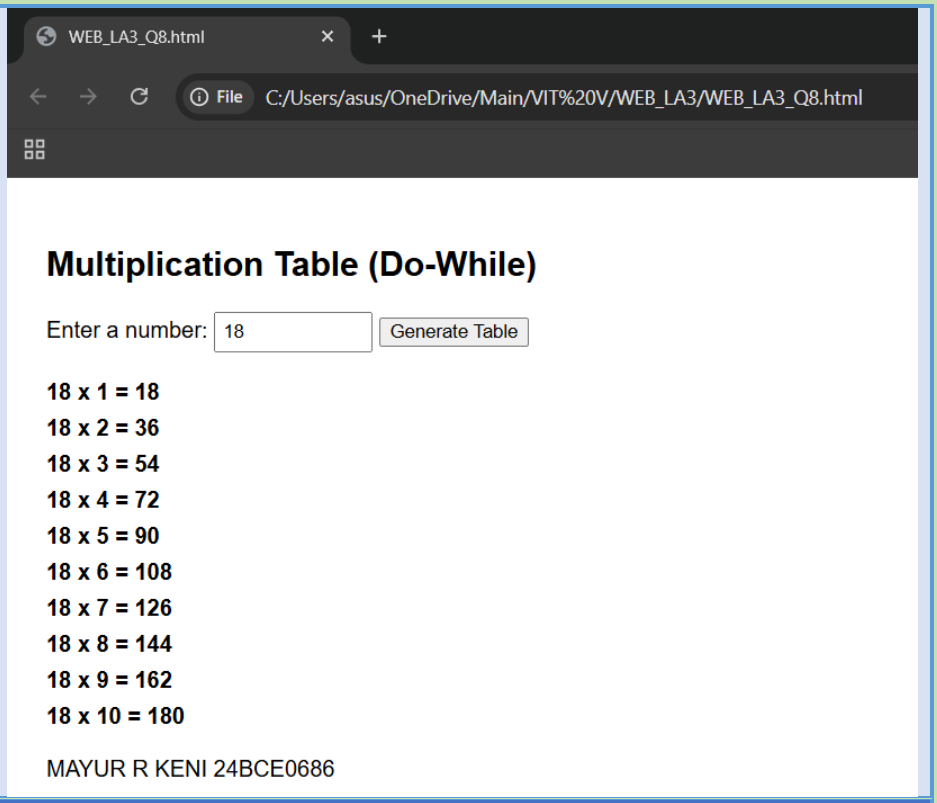
List of tags used

Tag/Property	Description
<input type="number">	Captures the base number for the multiplication table from the user.
do...while	A loop structure that executes the code block once before checking the condition.
i = 1	Initialization of the counter variable to start the table from 1.
*	The arithmetic operator used to calculate the product of the number and the counter.
 	HTML line break tag used within the JavaScript string to display each calculation on a new line.
.innerHTML	Property used to inject the full formatted multiplication table into the page.

Program

```
<!DOCTYPE html>
<html>
<head>
   <style>
      body { font-family: sans-serif; padding: 20px; }
      input { padding: 5px; width: 100px; }
      #tableOutput { margin-top: 15px; line-height: 1.6; font-weight: bold; }
   </style>
</head>
<body>
   <h2>Multiplication Table (Do-While)</h2>
   Enter a number: <input type="number" id="numInput">
   <button onclick="generateTable()">Generate Table</button>

   <div id="tableOutput"></div>
```

```
    <script>
    function generateTable() {
        var num =
parseInt(document.getElementById("numInput").value);
        var display = document.getElementById("tableOutput");
        var i = 1;
        var result = "";

        if (isNaN(num)) {
            display.innerHTML = "Please enter a valid number.";
            return;
        }

        do {
            var product = num * i;
            result += num + " x " + i + " = " + product + "<br>";
            i++;
        } while (i <= 10);

        display.innerHTML = result;
    }
    </script>
    <p>MAYUR R KENI 24BCE0686</p>
</body>
</html>
```

Output Screenshots	
Observations	This program highlights the unique behavior of the **do-while loop**, which ensures that the multiplication logic is processed at least once before the condition (i <= 10) is evaluated. By performing the multiplication calculation inside the loop and appending the results with the ** ** tag, the script builds a vertically formatted table. This demonstrates how JavaScript can handle mathematical operations and UI formatting simultaneously to present data clearly to the user.
References	

Assessment: Date:	Problem Statement-9
	Write a JavaScript program to display selected radio button value. Create three radio buttons for colors (Red, Green, Blue). Concepts: Radio button grouping using same name attribute Only one selection allowed in a group Accessing collection of elements by name Checking selected state using .checked property Looping through radio button collection

List of tags used

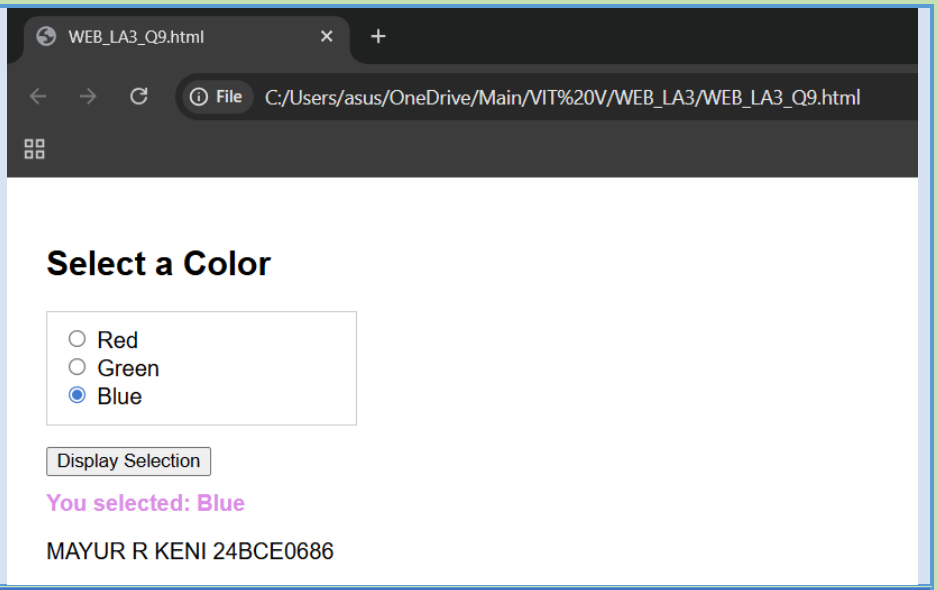
Tag/Property	Description
<input type="radio">	Creates a selection button where only one option in a group can be chosen.
name="color"	The attribute used to group radio buttons together; ensuring only one is selectable at a time.
getElementsByName()	A DOM method that returns a collection of all elements with the specified name.
for	A loop used to iterate through the collection of radio buttons to find the selected one.
.checked	A boolean property that returns true if a specific radio button is currently selected.
.value	The property that holds the data (e.g., "Red") associated with the selected radio button.

s

Program

```
<!DOCTYPE html>
<html>
<head>
  <style>
    body { font-family: sans-serif; padding: 20px; }
    .radio-group { margin-bottom: 15px; border: 1px solid #ccc; padding: 10px; width: 200px; }
    #result { margin-top: 10px; font-weight: bold; color: violet; }
  </style>
</head>
<body>
```

```
    <h2>Select a Color</h2>
    <div class="radio-group">
        <input type="radio" name="color" value="Red" id="r1"> <label
for="r1">Red</label><br>
        <input type="radio" name="color" value="Green" id="r2"> <label
for="r2">Green</label><br>
        <input type="radio" name="color" value="Blue" id="r3"> <label
for="r3">Blue</label>
    </div>

    <button onclick="checkSelection()">Display Selection</button>
    <p id="result"></p>

    <script>
    function checkSelection() {
        var colors = document.getElementsByName("color");
        var display = document.getElementById("result");
        var selectedValue = "";

        for (var i = 0; i < colors.length; i++) {
            if (colors[i].checked) {
                selectedValue = colors[i].value;
                break;
            }
        }

        if (selectedValue !== "") {
            display.innerHTML = "You selected: " + selectedValue;
        } else {
            display.innerHTML = "Please select a color.";
        }
    }
    </script>
    <p>MAYUR R KENI 24BCE0686</p>
</body>
</html>
```

Output Screenshots	
Observations	This program demonstrates how the name attribute creates a logical group for radio buttons, allowing only a single selection. By using document.getElementsByName(), JavaScript creates an array-like collection of these buttons. The for loop then checks each element's .checked state; once the selected button is found, the loop is terminated using the break statement for efficiency. This is a standard method for retrieving user choices from a set of mutually exclusive options.
References	

Assessment: Date:	Problem Statement-10
	Write a JavaScript program to count and display how many checkboxes are selected. Create four checkboxes for subjects. Concepts: Multiple selections allowed in checkboxes Building array of selected values Using push() to add elements to array Using join() to convert array to string Counter variable for counting selections

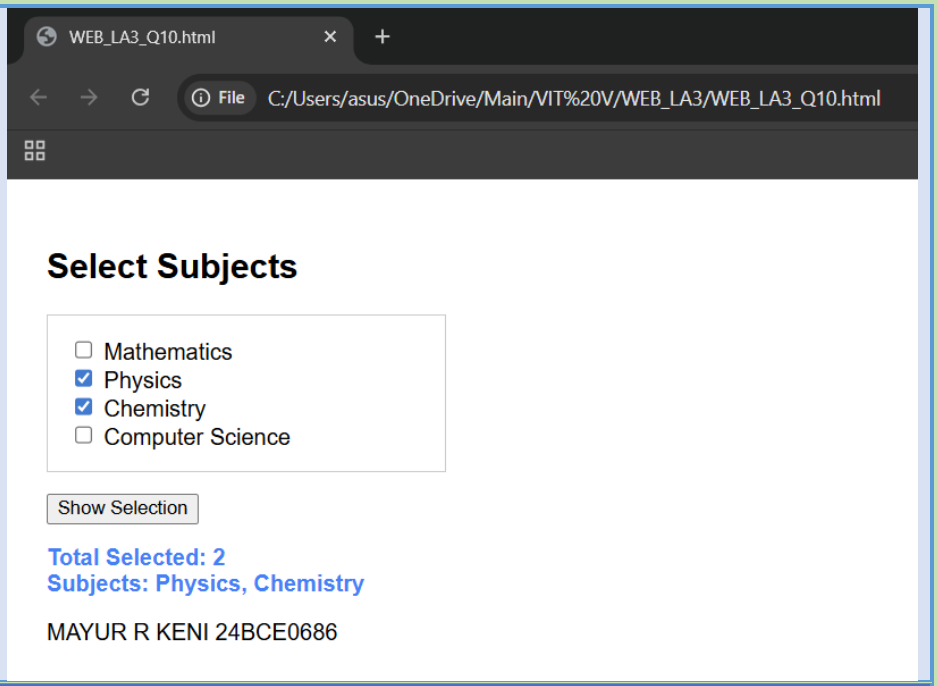
List of tags used

Tag/Property	Description
`<input type="checkbox">`	Allows the user to select multiple options from a set of subjects.
`getElementsByName()`	Retrieves all checkbox elements that share the same name attribute.
`Counter`	A variable used to track the total number of checked boxes.
`[] (Array)`	A collection used to store the values of only the selected subjects.
`push()`	An array method used to add the name of a selected subject into the array.
`join(", ")`	A method that converts the array into a single string, separated by commas for display.
`.checked`	A boolean property used in a loop to verify if a subject is selected.

Program

```
<!DOCTYPE html>
<html>
<head>
  <style>
    body { font-family: sans-serif; padding: 20px; }
    .subject-group { margin-bottom: 15px; border: 1px solid #ccc; padding: 15px; width: 250px; }
    #result { margin-top: 15px; font-weight: bold; color: #007bff; }
  </style>
</head>
<body>
```

```
    <h2>Select Subjects</h2>
    <div class="subject-group">
        <input type="checkbox" name="subj" value="Mathematics">
Mathematics<br>
        <input type="checkbox" name="subj" value="Physics">
Physics<br>
        <input type="checkbox" name="subj" value="Chemistry">
Chemistry<br>
        <input type="checkbox" name="subj" value="Computer
Science"> Computer Science
    </div>

    <button onclick="countSubjects()">Show Selection</button>

    <div id="result"></div>

    <script>
    function countSubjects() {
        var checkboxes = document.getElementsByName("subj");
        var selectedSubjects = [];
        var count = 0;

        for (var i = 0; i < checkboxes.length; i++) {
            if (checkboxes[i].checked) {
                selectedSubjects.push(checkboxes[i].value);
                count++;
            }
        }

        var display = document.getElementById("result");
        if (count > 0) {
            display.innerHTML = "Total Selected: " + count + "<br>" +
                                "Subjects: " + selectedSubjects.join(", ");
        } else {
            display.innerHTML = "No subjects selected.";
        }
    }
    </script>
    <p>MAYUR R KENI 24BCE0686</p>
</body>
</html>s
```

Output Screenshots	
Observations	This program illustrates the fundamental difference between radio buttons and checkboxes, as it allows for multiple selections. By using an array and the push() method, the script dynamically builds a list of user choices. The join() method is then used to format that data into a user-friendly string. This approach demonstrates efficient data collection and string manipulation within the DOM.
References	

Assessment: Date:	Problem Statement-11
	Write a JavaScript program to calculate simple interest using external JS file. Take principal, rate, and time as input from user. Concepts: External JavaScript file linking Mathematical calculation in external function Multiple input parameters handling Returning calculated result Formula implementation: SI = (P × R × T) / 100

List of tags used

Tag/Property	Description
<script src="script.js">	The tag used to link an external JavaScript file to the HTML document.
parseFloat()	Converts the string values from the input fields into decimal numbers for calculation.
SI = (P * R * T) / 100	The mathematical formula implemented to calculate simple interest.
return	A keyword used in functions to send the calculated result back to the caller.
toFixed(2)	A method used to format the result to two decimal places (standard for currency).

Program

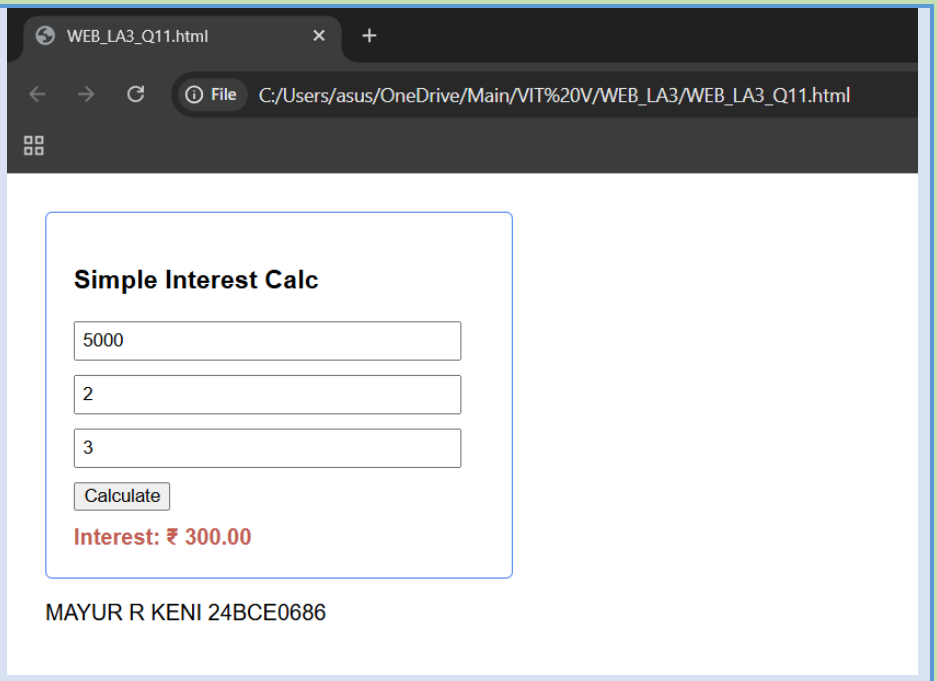
HTML:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
  <style>
    body { font-family: sans-serif; padding: 20px; }
    .calc-box { border: 1px solid #007bff; padding: 20px; width: 300px; border-radius: 5px; }
    input { margin-bottom: 10px; width: 90%; padding: 5px; }
    #result { font-weight: bold; color: #d9534f; margin-top: 10px; }
  </style>
</head>
<body>
  <div class="calc-box">
    <h3>Simple Interest Calc</h3>
```

```
        <input type="number" id="principal" placeholder="Principal (P)">
        <input type="number" id="rate" placeholder="Rate of Interest (R)">
        <input type="number" id="time" placeholder="Time in Years (T)">
        <button onclick="displaySI()">Calculate</button>
        <div id="result"></div>
    </div>

    <script src="interest.js"></script>
    <p>MAYUR R KENI 24BCE0686</p>
</body>
</html>
```

JS:

```
 function calculateSI(p, r, t) {
    var interest = (p * r * t) / 100;
    return interest;
}

function displaySI() {
    var p = parseFloat(document.getElementById("principal").value);
    var r = parseFloat(document.getElementById("rate").value);
    var t = parseFloat(document.getElementById("time").value);
    var output = document.getElementById("result");

    if (isNaN(p) || isNaN(r) || isNaN(t)) {
        output.innerHTML = "Please enter all values.";
        return;
    }

    var result = calculateSI(p, r, t);
    output.innerHTML = "Interest: ₹ " + result.toFixed(2);
}
```

Output Screenshots	
Observations	This program illustrates the best practice of separating **structure (HTML)** from **logic (JavaScript)** by using an external file. This makes the code more organized and easier to maintain. By passing the user inputs as parameters to a specialized calculation function (calculateSI), we demonstrate modular programming. The use of the **return** statement allows the logic to be reusable across different parts of an application.
References	

Assessment: Date:	Problem Statement-12
	Create a job skills form with multiple checkboxes. Use a for loop to validate that at least 3 skills are selected. Display the selected skills or an error message. Concepts: for loop with index Checkbox validation checkValidity() method Array length property String concatenation

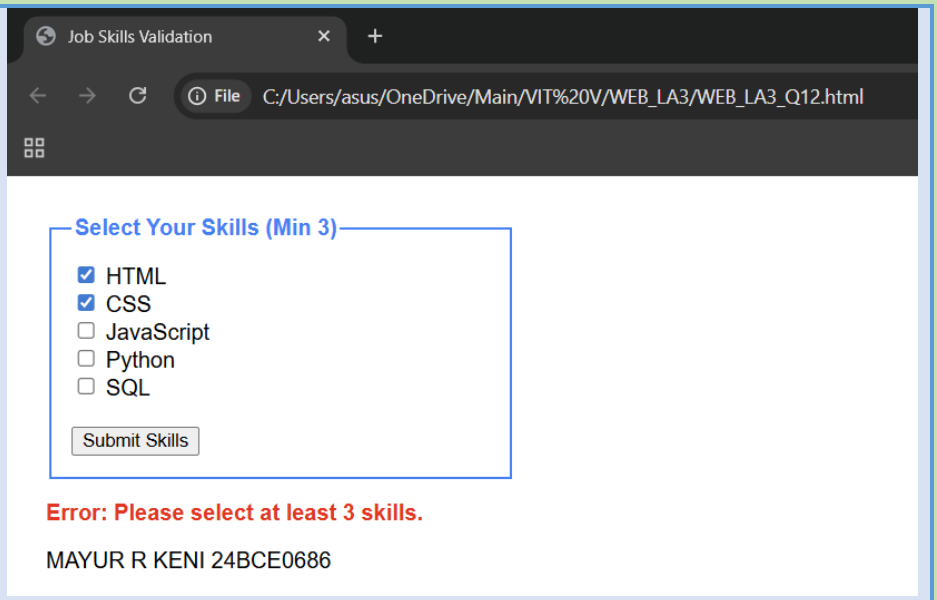
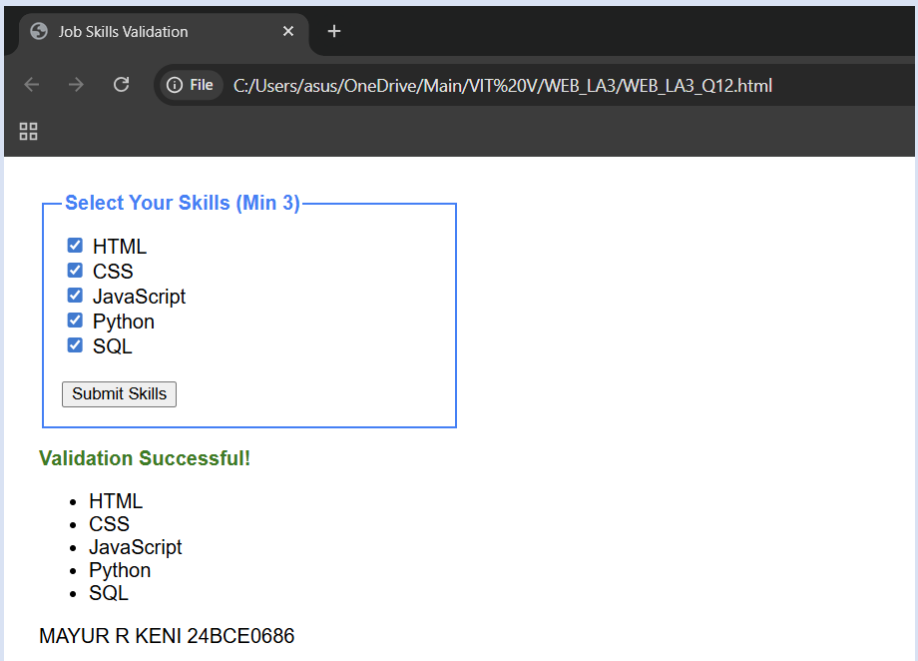
List of tags used

Tag/Property	Description
<form>	Collects the checkbox data for processing.
<fieldset>	Groups the skill options within a visual border.
getElementsByName()	Gets all checkboxes sharing the name "skills".
.length	Used in the loop to know how many checkboxes to check.
for	Iterates through each checkbox to see if it is selected.
+=	Concatenates the names of selected skills into a list.

Program

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Job Skills Validation</title>
    <style>
        body { font-family: sans-serif; padding: 20px; }
        fieldset { border: 2px solid #007bff; padding: 15px; width: 300px;
}
        legend { font-weight: bold; color: #007bff; }
        .error { color: red; font-weight: bold; }
        .success { color: green; font-weight: bold; }
        #displayMsg { margin-top: 15px; }
    </style>
    <script>
        function validateSkills() {
            var skillsCBs = document.getElementsByName("skills");
```

```
        var selectedCount = 0;
        var selectedValues = "";
        var msgArea = document.getElementById("displayMsg");

        for (let i = 0; i < skillsCBs.length; i++) {
            if (skillsCBs[i].checked) {
                selectedCount++;
                selectedValues += "<li>" + skillsCBs[i].value + "</li>";
            }
        }

        if (selectedCount >= 3) {
            msgArea.innerHTML = "<span class='success'>Validation
Successful!</span><ul>" + selectedValues + "</ul>";
        } else {
            msgArea.innerHTML = "<span class='error'>Error: Please
select at least 3 skills.</span>";
        }
    }
  </script>
</head>
<body>
  <form id="skillsForm">
    <fieldset>
        <legend>Select Your Skills (Min 3)</legend>
        <input type="checkbox" name="skills" value="HTML">
HTML<br>
        <input type="checkbox" name="skills" value="CSS">
CSS<br>
        <input type="checkbox" name="skills" value="JavaScript">
JavaScript<br>
        <input type="checkbox" name="skills" value="Python">
Python<br>
        <input type="checkbox" name="skills" value="SQL">
SQL<br><br>
        <button type="button" onclick="validateSkills()">Submit
Skills</button>
    </fieldset>
  </form>
  <div id="displayMsg"></div>
  <p>MAYUR R KENI 24BCE0686</p>
</body>
</html>
```

Output Screenshots	 
Observations	This program demonstrates how a for loop efficiently checks multiple elements to enforce a specific rule (selecting at least 3 items). By using the length property, the code remains flexible even if more checkboxes are added to the HTML later.
References	

Assessment: Date:	Problem Statement-13
	Create a product order form with multiple fieldsets. Use the modern for...of loop to calculate total price based on selected items from different categories. Concepts: for...of loop (modern JavaScript) Multiple fieldsets Dynamic calculation parseFloat() for price conversion

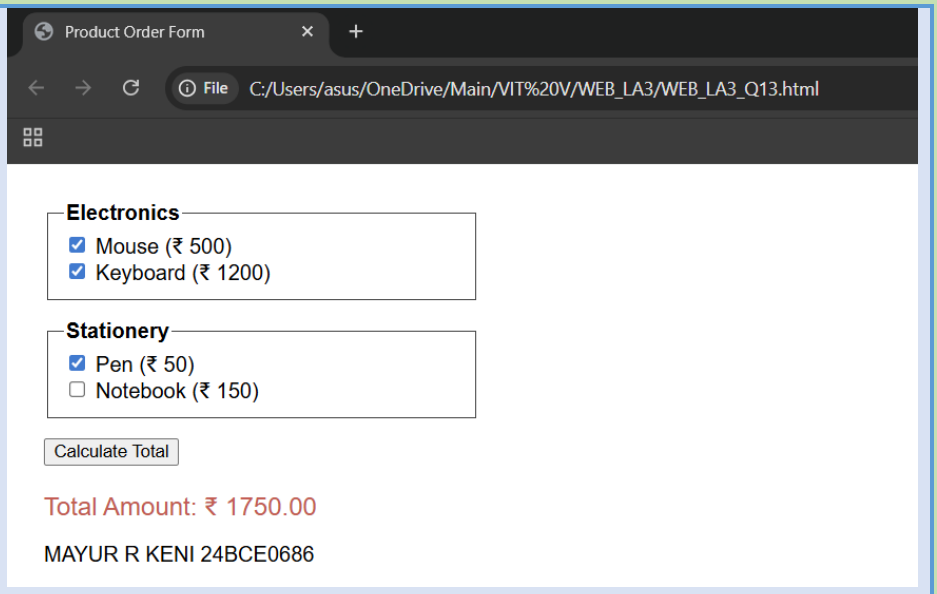
List of tags used

Tag/Property	Description
<fieldset>	Used to categorize products (e.g., Electronics, Stationery) into distinct visual groups.
for...of	A modern JavaScript loop used to iterate directly over the values of a collection (checkboxes).
data-price	A custom data attribute used to store the price of each item directly in the HTML tag.
parseFloat()	Converts the price string from the data attribute into a decimal number for calculation.
+=	Accumulates the individual prices into a single total variable.
.toFixed(2)	Formats the final total to show exactly two decimal places.

Program

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Product Order Form</title>
   <style>
      body { font-family: sans-serif; padding: 20px; }
      fieldset { border: 1px solid #444; margin-bottom: 15px; width: 300px; }
      legend { font-weight: bold; }
      #totalPrice { margin-top: 20px; font-size: 1.2em; color: #d9534f; }
   </style>
   <script>
```

```
        function calculateTotal() {
            var items = document.getElementsByName("product");
            var total = 0;

            for (let item of items) {
                if (item.checked) {
                    total += parseFloat(item.getAttribute("data-price"));
                }
            }

            document.getElementById("totalPrice").innerHTML = "Total
Amount: &#8377; " + total.toFixed(2);
        }
    </script>
</head>
<body>
    <form>
        <fieldset>
            <legend>Electronics</legend>
            <input type="checkbox" name="product" data-price="500">
Mouse (&#8377; 500)<br>
            <input type="checkbox" name="product" data-price="1200">
Keyboard (&#8377; 1200)
        </fieldset>

        <fieldset>
            <legend>Stationery</legend>
            <input type="checkbox" name="product" data-price="50"> Pen
(&#8377; 50)<br>
            <input type="checkbox" name="product" data-price="150">
Notebook (&#8377; 150)
        </fieldset>

        <button type="button" onclick="calculateTotal()">Calculate
Total</button>
    </form>

    <div id="totalPrice">Total Amount: &#8377; 0.00</div>
    <p>MAYUR R KENI 24BCE0686</p>
</body>
</html>
```

Output Screenshots	
Observations	This program demonstrates the efficiency of the for...of loop, which simplifies the code by removing the need for an index variable (i). By using data-price attributes, we keep the pricing logic tied directly to the HTML elements, making the parseFloat() operation straightforward. The use of multiple fieldsets improves user experience by organizing products into logical categories.
References	

Assessment: Date:	Problem Statement-14
	Create a text styling application. Use buttons to apply predefined CSS classes (italic, bold, small-caps) to a message using the className property. Concepts: className property Predefined CSS classes Multiple class assignment fieldset and legend for grouping.

List of tags used

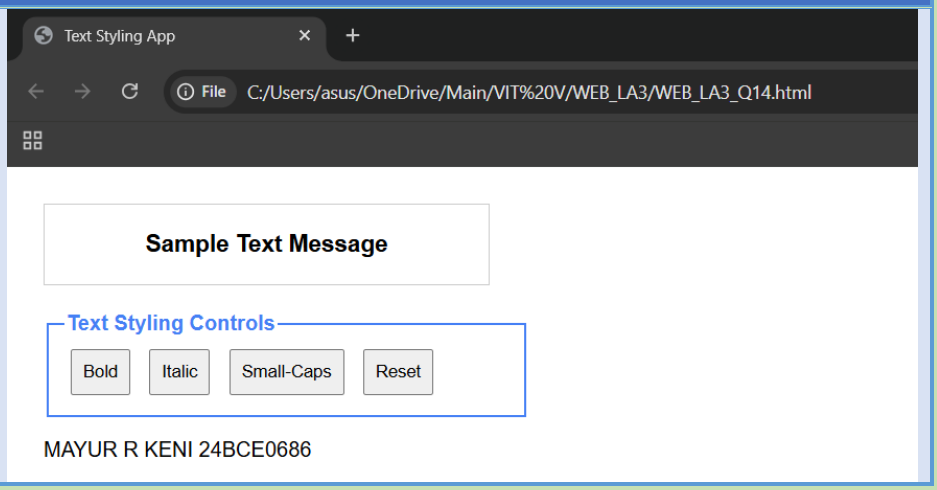
Tag/Property	Description
<fieldset> / <legend>	Groups the control buttons and provides a descriptive title for the styling panel.
.className	A JavaScript property used to set or change the entire class attribute of an element.
.bold / .italic / .caps	Custom CSS classes defined in the <style> block to handle specific text formatting.
String Assignment	The method of assigning a class name (e.g., element.className = "bold") via JavaScript.

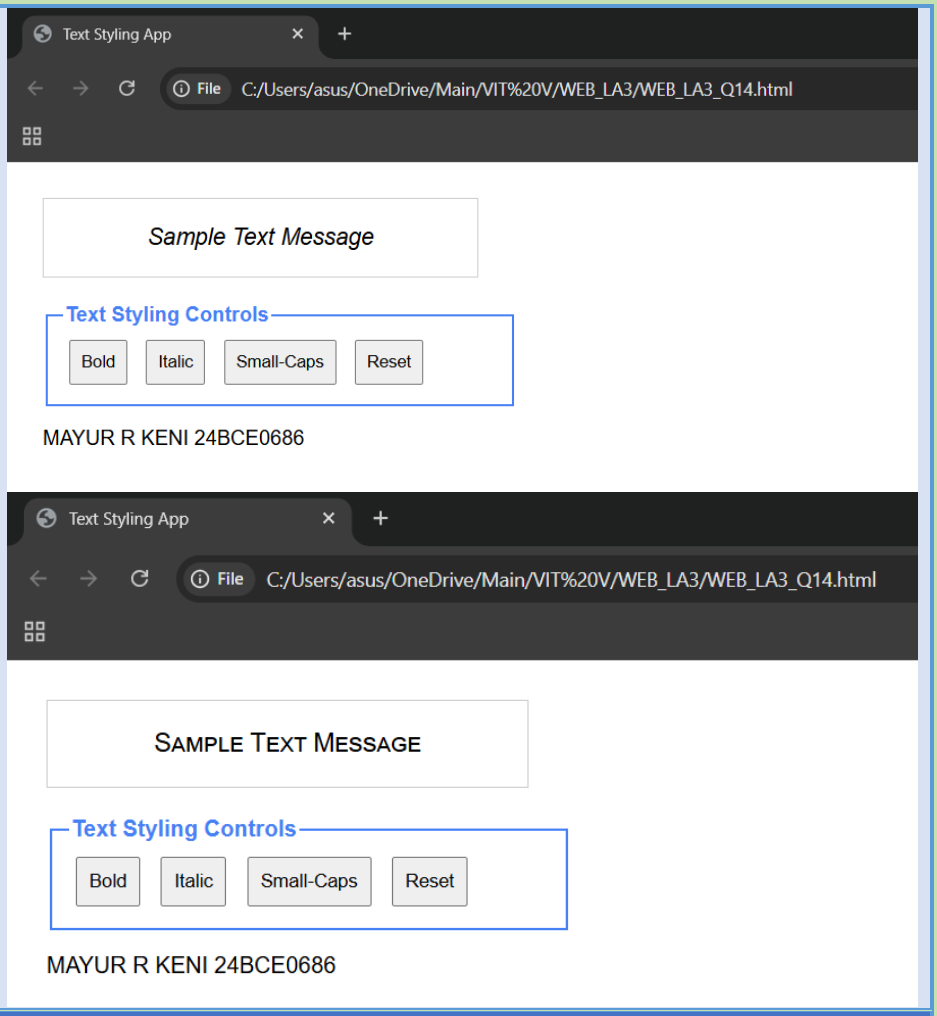
Program

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Text Styling App</title>
  <style>
    body { font-family: sans-serif; padding: 20px; }

    #textMessage {
      padding: 20px;
      border: 1px solid #ccc;
      margin-bottom: 20px;
      font-size: 18px;
      width: 300px;
      text-align: center;
    }

    /* Predefined CSS Classes */
    .bold-style { font-weight: bold; }
    .italic-style { font-style: italic; }
    .caps-style { font-variant: small-caps; }
```

```
        .reset-style { font-weight: normal; font-style: normal; font-variant: normal; }

        fieldset { width: 340px; border: 2px solid #007bff; }
        legend { font-weight: bold; color: #007bff; }
        button { margin: 5px; padding: 8px; cursor: pointer; }
    </style>
    <script>
        function applyStyle(styleName) {
            var msg = document.getElementById("textMessage");
            msg.className = styleName;
        }
    </script>
</head>
<body>

    <div id="textMessage">Sample Text Message</div>

    <fieldset>
        <legend>Text Styling Controls</legend>
        <button onclick="applyStyle('bold-style')">Bold</button>
        <button onclick="applyStyle('italic-style')">Italic</button>
        <button onclick="applyStyle('caps-style')">Small-Caps</button>
        <button onclick="applyStyle('reset-style')">Reset</button>
    </fieldset>

    <p>MAYUR R KENI 24BCE0686</p>
</body>
</html>
```

Output Screenshots





Observations	This program demonstrates how the className property serves as a bridge between JavaScript logic and CSS design. Instead of modifying individual inline styles (like style.fontWeight), we switch entire classes, which is more efficient for maintaining complex designs. The use of a fieldset ensures the user interface is organized and accessible.
References	

Assessment: Date:	Problem Statement-15
	Create a dynamic font customization tool. Use the style property to directly modify CSS properties (fontSize, color, backgroundColor) based on user input. Concepts: style property for inline CSS Dynamic property changes fieldset grouping Color picker input Range slider input

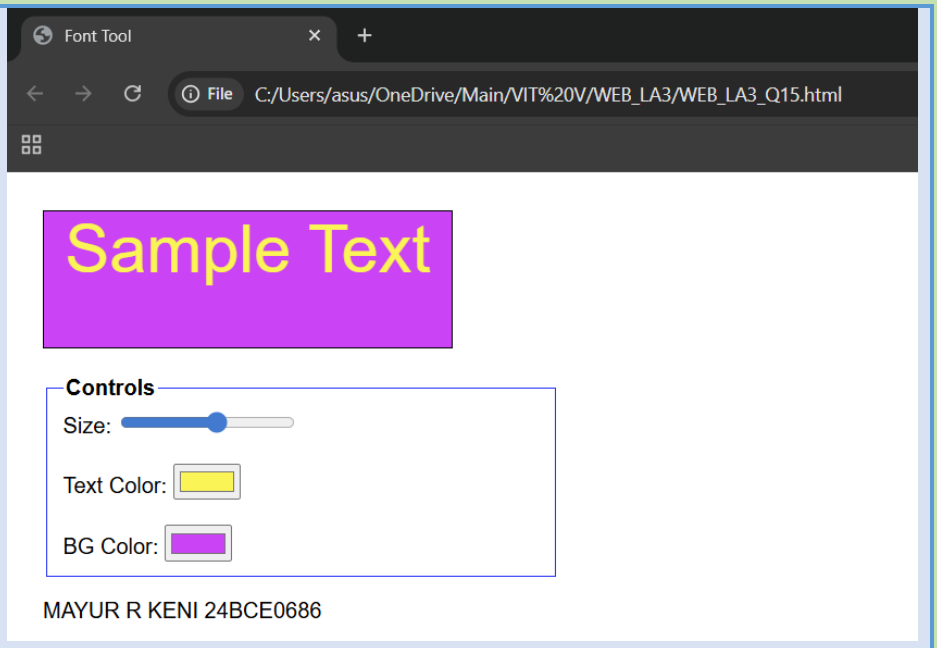
List of tags used

Tag/Property	Description
#previewBox	A unique ID selector used to target the specific div that displays the text.
<input type="range">	A slider used to select a number (10 to 80) for the font size.
<input type="color">	A picker used to select hex color codes for text and background.
.style	The JavaScript property used to modify inline CSS directly.
fontSize	Updates the size of the text (concatenated with "px").
oninput	An event that triggers the updateStyles() function immediately during user interaction.

Program

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Font Tool</title>
  <style>
    body { font-family: sans-serif; padding: 20px; }

    #previewBox {
      width: 300px;
      height: 100px;
        text-align: center;
      border: 1px solid black;
      margin-bottom: 20px;
    }
```

```
        fieldset { width: 350px; border: 1px solid blue; }
        legend { font-weight: bold; }
    </style>
    <script>
        function updateStyles() {
            var text = document.getElementById("previewBox");
            var size = document.getElementById("fontSize").value;
            var textColor = document.getElementById("textColor").value;
            var bgColor = document.getElementById("bgColor").value;

            text.style.fontSize = size + "px";
            text.style.color = textColor;
            text.style.backgroundColor = bgColor;
        }
    </script>
</head>
<body>

    <div id="previewBox">Sample Text</div>

    <fieldset>
        <legend>Controls</legend>

        Size: <input type="range" id="fontSize" min="10" max="80" value="20" oninput="updateStyles()"><br><br>

        Text Color: <input type="color" id="textColor" oninput="updateStyles()"><br><br>

        BG Color: <input type="color" id="bgColor" value="#ffffff" oninput="updateStyles()">

    </fieldset>

    <p>MAYUR R KENI 24BCE0686</p>
</body>
</html>
```

Output Screenshots	
Observations	This program functions as a **real-time customization interface**, allowing the user to visually manipulate text properties without writing code. By linking user-controlled inputs (sliders and color pickers) to the element's **style property**, the application demonstrates how JavaScript can provide immediate visual feedback. This is a practical example of creating **interactive UI tools** where the layout and design adapt dynamically based on user choices.
References	

Assessment : Date:	Problem Statement-16
	Create an overlapping image gallery where clicking an image brings it to the front by dynamically modifying its z-index using JavaScript. Concepts: z-index CSS property position: absolute Global variable for tracking top index ++ increment operator (prefix mode)

List of tags used

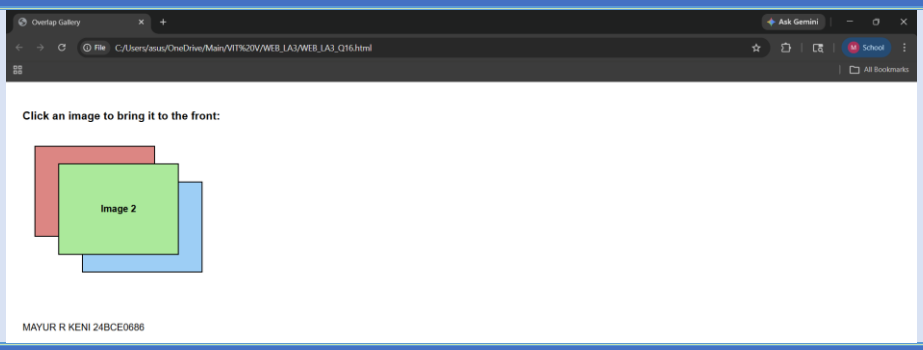
Tag/Property	Description
position: absolute	Essential for stacking elements on top of each other at specific coordinates.
z-index	The CSS property that determines the front-to-back order of overlapping elements.
Global Variable	var topZ = 1; is declared outside the function to maintain the stack count across multiple clicks.
++ (Prefix)	The increment operator used to increase the global counter before assigning it to the zIndex.
this	A keyword used in the onclick attribute to pass the clicked element itself into the function.

Program

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Overlap Gallery</title>
  <style>
    body { font-family: sans-serif; padding: 20px; }

    .gallery-container {
      position: relative;
      height: 300px;
    }

    .gallery-item {
      position: absolute;
      width: 200px;
      height: 150px;
      border: 2px solid black;
```

```
            cursor: pointer;
            text-align: center;
            line-height: 150px;
            font-weight: bold;
        }


        #img1 { top: 20px; left: 20px; background-color: lightcoral; }
        #img2 { top: 50px; left: 60px; background-color: lightgreen; }
        #img3 { top: 80px; left: 100px; background-color: lightskyblue; }
    </style>
    <script>
        var topZ = 1;

        function bringToFront(element) {
            element.style.zIndex = ++topZ;
        }
    </script>
</head>
<body>

    <h3>Click an image to bring it to the front:</h3>

    <div class="gallery-container">
        <div                    id="img1"                    class="gallery-item"
onclick="bringToFront(this)">Image 1</div>
        <div                    id="img2"                    class="gallery-item"
onclick="bringToFront(this)">Image 2</div>
        <div                    id="img3"                    class="gallery-item"
onclick="bringToFront(this)">Image 3</div>
    </div>

    <p>MAYUR R KENI 24BCE0686</p>
</body>
</html>
```

Output Screenshots



Observations	This program demonstrates how to manipulate the **stacking context** of a webpage. By using a global variable and the **prefix increment operator (++)**, the script ensures that every time an image is clicked, its z-index becomes higher than any other element currently on the screen. This allows for a dynamic "bring to front" behavior, showcasing how JavaScript can control 3D depth on a 2D screen.
References	